

MODELADO Y GESTIÓN DE BASE DE DATOS

TALLER FUNDAMENTOS DEL MODELADO DE BASE DE DATOS

PRESENTA:

BERNAL GORDILLO JUAN CAMILO

NEIVA – HUILA

COORPORACIÓN UNIVERSITARIA DEL HUILA – CORHUILA

SEPTIEMBRE DEL 2025

DESARROLLO

1) Fase de Análisis

El escenario que escogeré será el de un cine (FilmView). El cine pienso que es un escenario sencillo que me permitirá implementar de manera completa los fundamentos que hemos aprendido en este primer corte. Este cine estará definido de la siguiente manera:

- **Propósito**

Desarrollar un sistema que permita gestionar la información sobre las películas que se encuentran en cartelera, los clientes y los boletos vendidos dentro de un cine.

- **Alcance**

Cliente: En esta entidad se tendrá registro de los clientes que compran los boletos, contará con atributos que lo describan como: id_cliente (PK), nombre y correo; y tendrá una relación de 1:n con boletos (un cliente puede comprar muchos boletos, pero un boleto solo puede ser comprado por una sola persona).

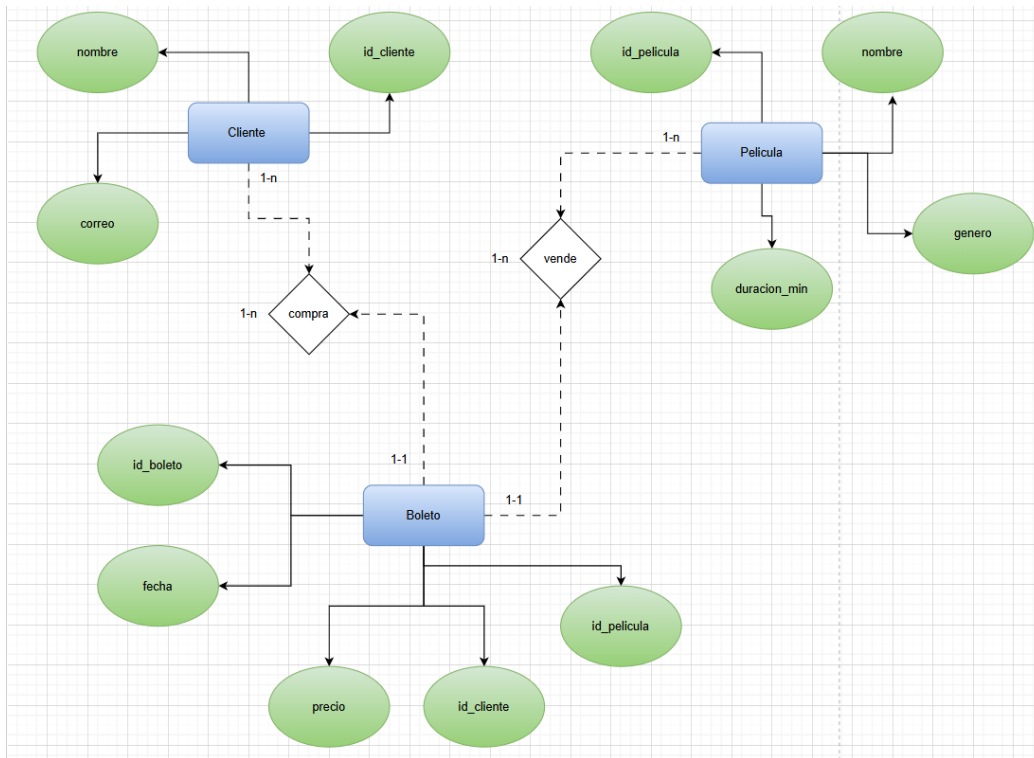
Película: En esta entidad se tendrá registro de cada película que se encuentra en cartelera, tendrá atributos como: id_pelicula (PK), nombre, genero, duración_min; y tendrá también una relación de 1:n con boletos (una película puede vender muchos boletos, pero un boleto solo puede venderse para ver una película).

Boleto: En esta entidad se obtendrá registro de cada venta generada, contará con atributos como: id_boleto (PK), fecha, precio, id_pelicula (FK) y id_cliente (FK). Tendrá relación 1:n con cliente y 1:n con película.

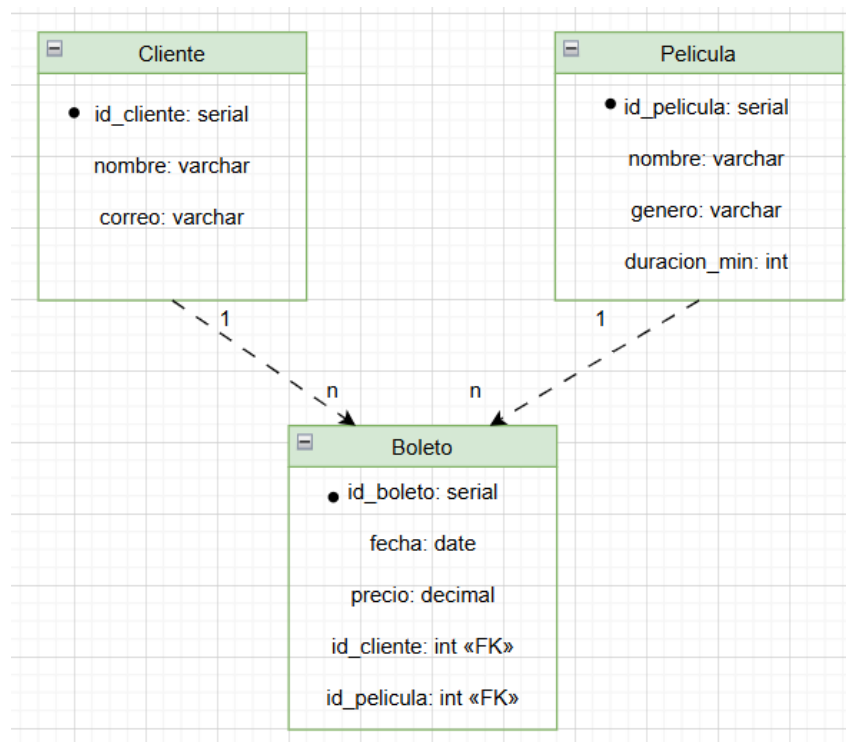
Nota: Se realiza la normalización con Boleto ya que sin esta tabla intermedia se formaría una relación de n:m, ya que una persona puede ver muchas películas y una película puede verla muchas personas.

2) Fase de Diseño

- **MER en Draw.io**



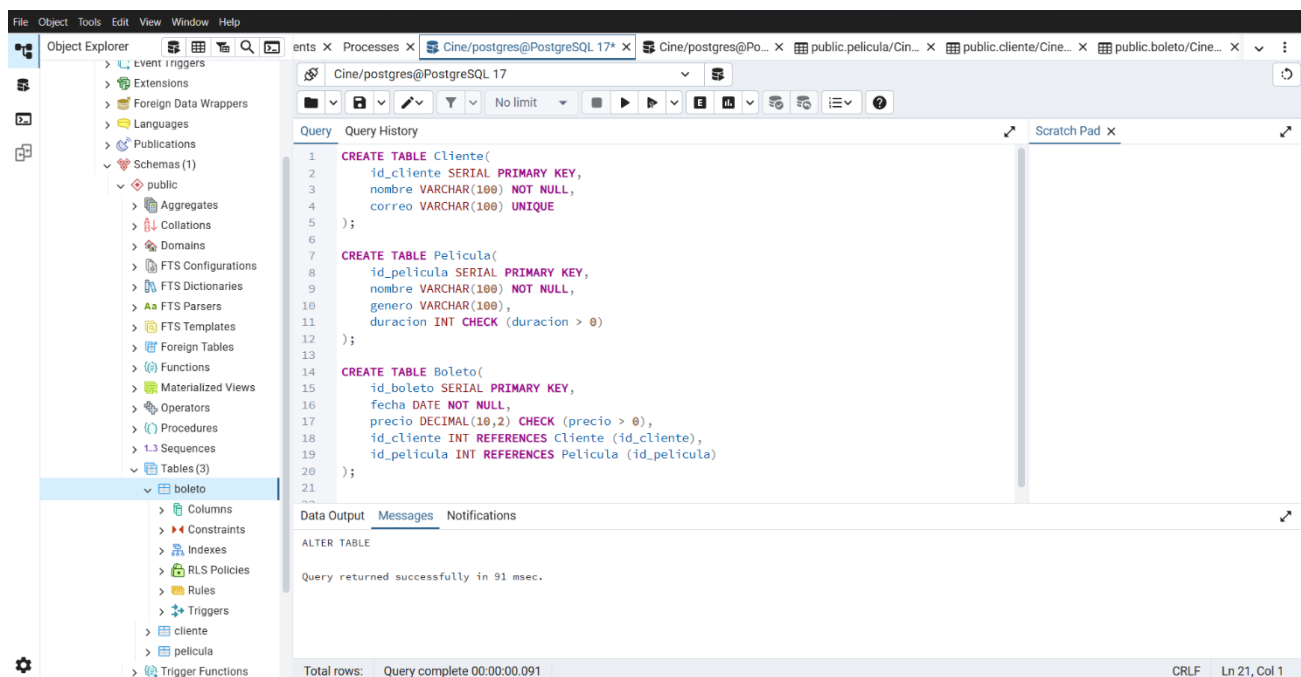
- **Transformación al Modelo Relacional (MR)**



- **Justificación:** Elegí organizar la información en tres tablas o entidades (Cliente, Película, Boleto), esto con el fin de que la información este bien organizada y no se repita. Cada una tienen atributos únicos que la identifican junto con su tipo de dato correcto para garantizar la consistencia de los datos. La tabla boleto la incluí porque ayuda a normalizar el ejercicio, pues ella nos ayuda a tener dos cardinalidades 1:n en lugar de una n:m. Esta a su vez, contiene llaves foráneas tanto de Cliente como de Película mediante los ids únicos de cada entidad, esto permite identificar que persona compró el boleto y que película va a ver, es como si fuera el corazón del ejercicio.

3) Fase de implementación

- **Creación de tablas con restricciones (PK, FK, UNIQUE, CHECK)**



Nota: Realice las tablas, pero por tema de claridad cambie el atributo duracion por duración_min para especificar que se mide en minutos, mediante el comando:

ALTER TABLE Pelicula

RENAME COLUMN duracion TO duracion_min;

- Inserción de datos de prueba

Cliente:

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure, with the 'cliente' table selected under the 'public' schema. The main query editor contains the following SQL query:

```
1 SELECT * FROM public.cliente
2 ORDER BY id_cliente ASC
```

The 'Data Output' tab at the bottom shows the results of the query, displaying 5 rows. The status bar indicates 'Total rows: 5' and 'Query complete 00:00:00.222'.

id_cliente	nombre	correo
12312212	Mauricio Quiñones	Mauri.Quiñones@gmail.com
12903876	Alejandro Ramirez	ninja66@gmail.com
23133212	Adriana Quintero	Quintero_Adri@gmail.com
24526390	Ester Castro	CEster@gmail.com
55678982	Maykol Vanegas	progamer@gmail.com

Película:

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure, with the 'película' table selected under the 'public' schema. The main query editor contains the following SQL query:

```
1 SELECT * FROM public.película
2 ORDER BY id_película ASC
```

The 'Data Output' tab at the bottom shows the results of the query, displaying 5 rows. The status bar indicates 'Total rows: 5' and 'Query complete 00:00:00.209'.

id_película	nombre	genero	duracion_min
238907	Los tipos malos 2	Animación	105
1231489	Sin Oxígeno	Suspense	93
34592213	Conjuro 4	Terror	120
78653401	Weapons	Terror	128
89076314	Acosada	Drama	111

Boleto:

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the 'Object Explorer' with the 'public' schema expanded, showing tables 'cliente', 'pelicula', and 'Boleto'. The 'Boleto' table is selected. The main pane shows a query window with the following SQL:

```
1 SELECT * FROM public.boleto
2 ORDER BY id_boleto ASC
```

The 'Data Output' pane shows the results of the query, displaying 5 rows. The columns are: id_boleto (integer), fecha (date), precio (numeric), id_cliente (integer), and id_pelicula (integer).

id_boleto	fecha	precio	id_cliente	id_pelicula
1	2025-09-24	2300.45	23133212	238907
2	2025-09-11	2300.45	24526390	238907
3	2025-08-23	2600.99	55678982	34592213
4	2025-06-01	2500.55	12903876	89076314
5	2025-06-01	2500.55	12903876	78653401

Total rows: 5 Query complete 00:00:00.225

4) Fase de pruebas:

- Consultas básicas

Select:

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the 'Object Explorer' with the 'public' schema expanded, showing tables 'cliente', 'pelicula', and 'Boleto'. The 'Boleto' table is selected. The main pane shows a query window with the following SQL:

```
9 nombre VARCHAR(100) NOT NULL,
10 genero VARCHAR(100),
11 duracion INT CHECK (duracion > 0)
12 );
13
14 CREATE TABLE Boleto(
15 id_boleto SERIAL PRIMARY KEY,
16 fecha DATE NOT NULL,
17 precio DECIMAL(10,2) CHECK (precio > 0),
18 id_cliente INT REFERENCES Cliente (id_cliente),
19 id_pelicula INT REFERENCES Pelicula (id_pelicula)
20 );
21
22 SELECT nombre, id_pelicula FROM Pelicula;
23
```

The 'Data Output' pane shows the results of the query, displaying 5 rows. The columns are: nombre (character varying) and id_pelicula (integer).

nombre	id_pelicula
Conjuro 4	34592213
Los tipos malos 2	238907
Sin Oxígeno	1231489
Acosada	89076314
Weapons	78653401

Total rows: 5 Query complete 00:00:00.342

Insert:

The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database structure, with the 'public' schema selected and the 'boleto' table highlighted under 'Tables (3)'. The main query editor contains the following SQL code:

```
12 );
13
14 CREATE TABLE Boleto(
15     id_boleto SERIAL PRIMARY KEY,
16     fecha DATE NOT NULL,
17     precio DECIMAL(10,2) CHECK (precio > 0),
18     id_cliente INT REFERENCES Cliente (id_cliente),
19     id_pelicula INT REFERENCES Pelicula (id_pelicula)
20 );
21
22 INSERT INTO Boleto(id_boleto, fecha, precio, id_cliente, id_pelicula)
23 VALUES (422214, '1/06/25', 2500.55, 12903876, 89076314);
24 SELECT nombre, id_pelicula FROM Pelicula;
25 DELETE FROM Boleto WHERE id_boleto=7782920;
26
```

The 'Data Output' tab shows the result of the INSERT statement:

```
INSERT 0 1
```

Below the output, a message states: 'Query returned successfully in 77 msec.' A green status bar at the bottom right confirms: '✓ Query returned successfully in 77 msec. X'.

Delete:

The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database structure, with the 'public' schema selected and the 'boleto' table highlighted under 'Tables (3)'. The main query editor contains the following SQL code:

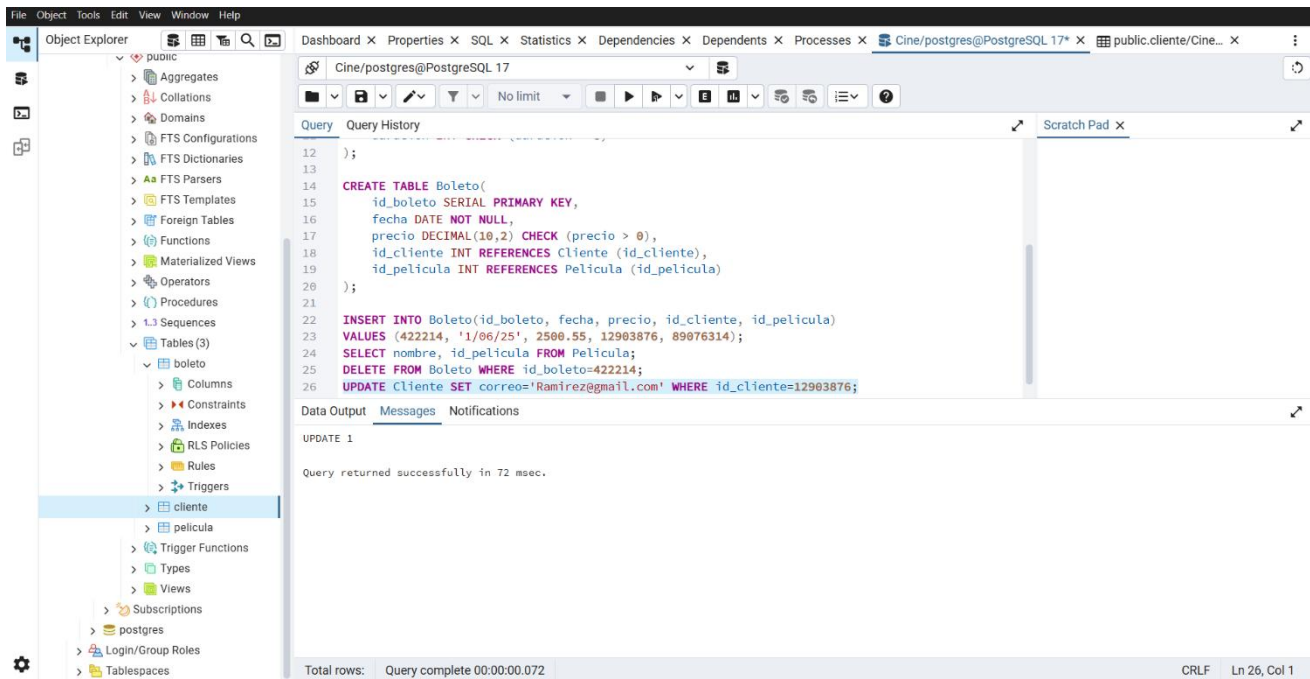
```
12 );
13
14 CREATE TABLE Boleto(
15     id_boleto SERIAL PRIMARY KEY,
16     fecha DATE NOT NULL,
17     precio DECIMAL(10,2) CHECK (precio > 0),
18     id_cliente INT REFERENCES Cliente (id_cliente),
19     id_pelicula INT REFERENCES Pelicula (id_pelicula)
20 );
21
22 INSERT INTO Boleto(id_boleto, fecha, precio, id_cliente, id_pelicula)
23 VALUES (422214, '1/06/25', 2500.55, 12903876, 89076314);
24 SELECT nombre, id_pelicula FROM Pelicula;
25 DELETE FROM Boleto WHERE id_boleto=422214;
26
```

The 'Data Output' tab shows the result of the DELETE statement:

```
DELETE 1
```

Below the output, a message states: 'Query returned successfully in 76 msec.' A green status bar at the bottom right confirms: '✓ Query returned successfully in 76 msec. X'.

Update:



- **DBA y Diseñador en el Proyecto**

DBA:

El DBA es el que se encarga de la parte técnica, de que la base que ya está funcione de manera segura y optima, es decir, es el que se encarga de que las bases sigan funcionando con seguridad, disponibilidad e integridad.

En el proyecto diría que tendría las siguientes funciones y responsabilidades:

- 1) Crear las tablas en el programa PostgreSQL con las restricciones necesarias (PK, FK, CHECK, UNIQUE), junto con su inserción de datos correspondiente (en este caso 5 registros por lo menos).
- 2) Realizar las consultas y mirar que todo esté funcionando correctamente y corregir por si hay errores. Así como documentar los procesos.

Diseñador:

El diseñador se encarga de analizar y estructurar la base de datos antes de pasar a la creación, es decir, que este da la base en si sobre lo cual trabajar, no solamente en este proyecto sino en todos los demás.

En el proyecto que realicé diría que tendría las siguientes funciones y responsabilidades:

- 1) Identificar las entidades que íbamos a manejar y definir los atributos de cada una, así como las relaciones que se iban a establecer.
- 2) Crear el MER y transformarlo al modelo relacional usando las reglas de transformación (MR).
- 3) Justificar por qué se hizo el modelado de la forma en que se hizo.

En sí, diría que el diseñador se encargaría de las fases de análisis y diseño en este proyecto, en cambio el DBA se encargaría de las fases de implementación y prueba. En este proyecto se puede evidenciar correctamente lo importantes que son tanto el diseñador y el administrador, ya que el diseñador se encarga de sentar las bases y el DBA se encarga de que estas sigan funcionando correctamente (se complementan).